

```

10
11 ArrayList<Integer> convertIntToSet(int x, ArrayList<Integer> set) {
12     ArrayList<Integer> subset = new ArrayList<Integer>();
13     int index = 0;
14     for (int k = x; k > 0; k >>= 1) {
15         if ((k & 1) == 1) {
16             subset.add(set.get(index));
17         }
18         index++;
19     }
20     return subset;
21 }

```

There's nothing substantially better or worse about this solution compared to the first one.

8.5 Recursive Multiply: Write a recursive function to multiply two positive integers without using the `*` operator (or `/` operator). You can use addition, subtraction, and bit shifting, but you should minimize the number of those operations.

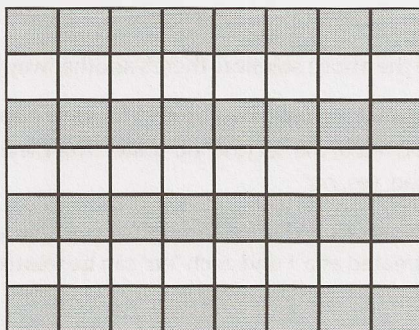
pg 135

SOLUTION

Let's pause for a moment and think about what it means to do multiplication.

This is a good approach for a lot of interview questions. It's often useful to think about what it really means to do something, even when it's pretty obvious.

We can think about multiplying 8×7 as doing $8+8+8+8+8+8+8$ (or adding 7 eight times). We can also think about it as the number of squares in an 8×7 grid.



Solution #1

How would we count the number of squares in this grid? We could just count each cell. That's pretty slow, though.

Alternatively, we could count half the squares and then double it (by adding this count to itself). To count half the squares, we repeat the same process.

Of course, this "doubling" only works if the number is in fact even. When it's not even, we need to do the counting/summing from scratch.

```

1 int minProduct(int a, int b) {
2     int bigger = a < b ? b : a;

```

```

3     int smaller = a < b ? a : b;
4     return minProductHelper(smaller, bigger);
5 }
6
7 int minProductHelper(int smaller, int bigger) {
8     if (smaller == 0) { // 0 x bigger = 0
9         return 0;
10    } else if (smaller == 1) { // 1 x bigger = bigger
11        return bigger;
12    }
13
14    /* Compute half. If uneven, compute other half. If even, double it. */
15    int s = smaller >> 1; // Divide by 2
16    int side1 = minProduct(s, bigger);
17    int side2 = side1;
18    if (smaller % 2 == 1) {
19        side2 = minProductHelper(smaller - s, bigger);
20    }
21
22    return side1 + side2;
23 }

```

Can we do better? Yes.

Solution #2

If we observe how the recursion operates, we'll notice that we have duplicated work. Consider this example:

```

minProduct(17, 23)
    minProduct(8, 23)
        minProduct(4, 23) * 2
        ...
    + minProduct(9, 23)
        minProduct(4, 23)
        ...
    + minProduct(5, 23)
        ...

```

The second call to `minProduct(4, 23)` is unaware of the prior call, and so it repeats the same work. We should cache these results.

```

1 int minProduct(int a, int b) {
2     int bigger = a < b ? b : a;
3     int smaller = a < b ? a : b;
4
5     int memo[] = new int[smaller + 1];
6     return minProduct(smaller, bigger, memo);
7 }
8
9 int minProduct(int smaller, int bigger, int[] memo) {
10    if (smaller == 0) {
11        return 0;
12    } else if (smaller == 1) {
13        return bigger;
14    } else if (memo[smaller] > 0) {
15        return memo[smaller];
16    }
17
18    /* Compute half. If uneven, compute other half. If even, double it. */

```

```
19  int s = smaller >> 1; // Divide by 2
20  int side1 = minProduct(s, bigger, memo); // Compute half
21  int side2 = side1;
22  if (smaller % 2 == 1) {
23      side2 = minProduct(smaller - s, bigger, memo);
24  }
25
26  /* Sum and cache.*/
27  memo[smaller] = side1 + side2;
28  return memo[smaller];
29 }
```

We can still make this a bit faster.

Solution #3

One thing we might notice when we look at this code is that a call to `minProduct` on an even number is much faster than one on an odd number. For example, if we call `minProduct(30, 35)`, then we'll just do `minProduct(15, 35)` and double the result. However, if we do `minProduct(31, 35)`, then we'll need to call `minProduct(15, 35)` and `minProduct(16, 35)`.

This is unnecessary. Instead, we can do:

```
minProduct(31, 35) = 2 * minProduct(15, 35) + 35
```

After all, since $31 = 2 \cdot 15 + 1$, then $31 \times 35 = 2 \cdot 15 \cdot 35 + 35$.

The logic in this final solution is that, on even numbers, we just divide `smaller` by 2 and double the result of the recursive call. On odd numbers, we do the same, but then we also add `bigger` to this result.

In doing so, we have an unexpected “win.” Our `minProduct` function just recurses straight downwards, with increasingly small numbers each time. It will never repeat the same call, so there's no need to cache any information.

```
1  int minProduct(int a, int b) {
2      int bigger = a < b ? b : a;
3      int smaller = a < b ? a : b;
4      return minProductHelper(smaller, bigger);
5  }
6
7  int minProductHelper(int smaller, int bigger) {
8      if (smaller == 0) return 0;
9      else if (smaller == 1) return bigger;
10
11     int s = smaller >> 1; // Divide by 2
12     int halfProd = minProductHelper(s, bigger);
13
14     if (smaller % 2 == 0) {
15         return halfProd + halfProd;
16     } else {
17         return halfProd + halfProd + bigger;
18     }
19 }
```

This algorithm will run in $O(\log s)$ time, where s is the smaller of the two numbers.

8.6 Towers of Hanoi: In the classic problem of the Towers of Hanoi, you have 3 towers and N disks of different sizes which can slide onto any tower. The puzzle starts with disks sorted in ascending order of size from top to bottom (i.e., each disk sits on top of an even larger one). You have the following constraints:

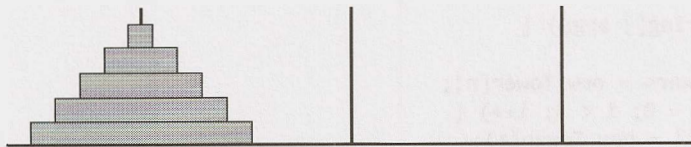
- (1) Only one disk can be moved at a time.
- (2) A disk is slid off the top of one tower onto another tower.
- (3) A disk cannot be placed on top of a smaller disk.

Write a program to move the disks from the first tower to the last using Stacks.

pg 135

SOLUTION

This problem sounds like a good candidate for the Base Case and Build approach.



Let's start with the smallest possible example: $n = 1$.

Case $n = 1$. Can we move Disk 1 from Tower 1 to Tower 3? Yes.

1. We simply move Disk 1 from Tower 1 to Tower 3.

Case $n = 2$. Can we move Disk 1 and Disk 2 from Tower 1 to Tower 3? Yes.

1. Move Disk 1 from Tower 1 to Tower 2
2. Move Disk 2 from Tower 1 to Tower 3
3. Move Disk 1 from Tower 2 to Tower 3

Note how in the above steps, Tower 2 acts as a buffer, holding a disk while we move other disks to Tower 3.

Case $n = 3$. Can we move Disk 1, 2, and 3 from Tower 1 to Tower 3? Yes.

1. We know we can move the top two disks from one tower to another (as shown earlier), so let's assume we've already done that. But instead, let's move them to Tower 2.
2. Move Disk 3 to Tower 3.
3. Move Disk 1 and Disk 2 to Tower 3. We already know how to do this—just repeat what we did in Step 1.

Case $n = 4$. Can we move Disk 1, 2, 3 and 4 from Tower 1 to Tower 3? Yes.

1. Move Disks 1, 2, and 3 to Tower 2. We know how to do that from the earlier examples.
2. Move Disk 4 to Tower 3.
3. Move Disks 1, 2 and 3 back to Tower 3.

Remember that the labels of Tower 2 and Tower 3 aren't important. They're equivalent towers. So, moving disks to Tower 3 with Tower 2 serving as a buffer is equivalent to moving disks to Tower 2 with Tower 3 serving as a buffer.

This approach leads to a natural recursive algorithm. In each part, we are doing the following steps, outlined below with pseudocode:

```
1  moveDisks(int n, Tower origin, Tower destination, Tower buffer) {
2      /* Base case */
3      if (n <= 0) return;
4
5      /* move top n - 1 disks from origin to buffer, using destination as a buffer. */
6      moveDisks(n - 1, origin, buffer, destination);
7
8      /* move top from origin to destination
9      moveTop(origin, destination);
10
11     /* move top n - 1 disks from buffer to destination, using origin as a buffer. */
12     moveDisks(n - 1, buffer, destination, origin);
13 }
```

The following code provides a more detailed implementation of this algorithm, using concepts of object-oriented design.

```
1  void main(String[] args) {
2      int n = 3;
3      Tower[] towers = new Tower[n];
4      for (int i = 0; i < 3; i++) {
5          towers[i] = new Tower(i);
6      }
7
8      for (int i = n - 1; i >= 0; i--) {
9          towers[0].add(i);
10     }
11     towers[0].moveDisks(n, towers[2], towers[1]);
12 }
13
14 class Tower {
15     private Stack<Integer> disks;
16     private int index;
17     public Tower(int i) {
18         disks = new Stack<Integer>();
19         index = i;
20     }
21
22     public int index() {
23         return index;
24     }
25
26     public void add(int d) {
27         if (!disks.isEmpty() && disks.peek() <= d) {
28             System.out.println("Error placing disk " + d);
29         } else {
30             disks.push(d);
31         }
32     }
33
34     public void moveTopTo(Tower t) {
35         int top = disks.pop();
36         t.add(top);
37     }
38 }
```

```

39 public void moveDisks(int n, Tower destination, Tower buffer) {
40     if (n > 0) {
41         moveDisks(n - 1, buffer, destination);
42         moveTopTo(destination);
43         buffer.moveDisks(n - 1, destination, this);
44     }
45 }
46 }

```

Implementing the towers as their own objects is not strictly necessary, but it does help to make the code cleaner in some respects.

8.7 Permutations without Dups: Write a method to compute all permutations of a string of unique characters.

pg 135

SOLUTION

Like in many recursive problems, the Base Case and Build approach will be useful. Assume we have a string S represented by the characters $a_1 a_2 \dots a_n$.

Approach 1: Building from permutations of first $n-1$ characters.

Base Case: permutations of first character substring

The only permutation of a_1 is the string a_1 . So:

$$P(a_1) = a_1$$

Case: permutations of $a_1 a_2$

$$P(a_1 a_2) = a_1 a_2 \text{ and } a_2 a_1$$

Case: permutations of $a_1 a_2 a_3$

$$P(a_1 a_2 a_3) = a_1 a_2 a_3, a_1 a_3 a_2, a_2 a_1 a_3, a_2 a_3 a_1, a_3 a_1 a_2, a_3 a_2 a_1,$$

Case: permutations of $a_1 a_2 a_3 a_4$

This is the first interesting case. How can we generate permutations of $a_1 a_2 a_3 a_4$ from $a_1 a_2 a_3$?

Each permutation of $a_1 a_2 a_3 a_4$ represents an ordering of $a_1 a_2 a_3$. For example, $a_2 a_4 a_1 a_3$ represents the order $a_2 a_1 a_3$.

Therefore, if we took all the permutations of $a_1 a_2 a_3$ and added a_4 into all possible locations, we would get all permutations of $a_1 a_2 a_3 a_4$.

```

a1a2a3 -> a4a1a2a3, a1a4a2a3, a1a2a4a3, a1a2a3a4
a1a3a2 -> a4a1a3a2, a1a4a3a2, a1a3a4a2, a1a3a2a4
a3a1a2 -> a4a3a1a2, a3a4a1a2, a3a1a4a2, a3a1a2a4
a2a1a3 -> a4a2a1a3, a2a4a1a3, a2a1a4a3, a2a1a3a4
a2a3a1 -> a4a2a3a1, a2a4a3a1, a2a3a4a1, a2a3a1a4
a3a2a1 -> a4a3a2a1, a3a4a2a1, a3a2a4a1, a3a2a1a4

```

We can now implement this algorithm recursively.

```

1 ArrayList<String> getPerms(String str) {
2     if (str == null) return null;
3
4     ArrayList<String> permutations = new ArrayList<String>();
5     if (str.length() == 0) { // base case
6         permutations.add("");

```

```

7         return permutations;
8     }
9
10    char first = str.charAt(0); // get the first char
11    String remainder = str.substring(1); // remove the first char
12    ArrayList<String> words = getPerms(remainder);
13    for (String word : words) {
14        for (int j = 0; j <= word.length(); j++) {
15            String s = insertCharAt(word, first, j);
16            permutations.add(s);
17        }
18    }
19    return permutations;
20 }
21
22 /* Insert char c at index i in word. */
23 String insertCharAt(String word, char c, int i) {
24     String start = word.substring(0, i);
25     String end = word.substring(i);
26     return start + c + end;
27 }

```

Approach 2: Building from permutations of all n-1 character substrings.

Base Case: single-character strings

The only permutation of a_1 is the string a_1 . So:

$$P(a_1) = a_1$$

Case: two-character strings

$$P(a_1a_2) = a_1a_2 \text{ and } a_2a_1.$$

$$P(a_2a_3) = a_2a_3 \text{ and } a_3a_2.$$

$$P(a_1a_3) = a_1a_3 \text{ and } a_3a_1.$$

Case: three-character strings

Here is where the cases get more interesting. How can we generate all permutations of three-character strings, such as $a_1a_2a_3$, given the permutations of two-character strings?

Well, in essence, we just need to “try” each character as the first character and then append the permutations.

$$\begin{aligned}
 P(a_1a_2a_3) &= \{a_1 + P(a_2a_3)\} + \{a_2 + P(a_1a_3)\} + \{a_3 + P(a_1a_2)\} \\
 \{a_1 + P(a_2a_3)\} &\rightarrow a_1a_2a_3, a_1a_3a_2 \\
 \{a_2 + P(a_1a_3)\} &\rightarrow a_2a_1a_3, a_2a_3a_1 \\
 \{a_3 + P(a_1a_2)\} &\rightarrow a_3a_1a_2, a_3a_2a_1
 \end{aligned}$$

Now that we can generate all permutations of three-character strings, we can use this to generate permutations of four-character strings.

$$P(a_1a_2a_3a_4) = \{a_1 + P(a_2a_3a_4)\} + \{a_2 + P(a_1a_3a_4)\} + \{a_3 + P(a_1a_2a_4)\} + \{a_4 + P(a_1a_2a_3)\}$$

This is now a fairly straightforward algorithm to implement.

```

1 ArrayList<String> getPerms(String remainder) {
2     int len = remainder.length();
3     ArrayList<String> result = new ArrayList<String>();
4
5     /* Base case. */

```

```

6     if (len == 0) {
7         result.add(""); // Be sure to return empty string!
8         return result;
9     }
10
11
12     for (int i = 0; i < len; i++) {
13         /* Remove char i and find permutations of remaining chars.*/
14         String before = remainder.substring(0, i);
15         String after = remainder.substring(i + 1, len);
16         ArrayList<String> partials = getPerms(before + after);
17
18         /* Prepend char i to each permutation.*/
19         for (String s : partials) {
20             result.add(remainder.charAt(i) + s);
21         }
22     }
23
24     return result;
25 }

```

Alternatively, instead of passing the permutations back up the stack, we can push the prefix down the stack. When we get to the bottom (base case), `prefix` holds a full permutation.

```

1  ArrayList<String> getPerms(String str) {
2      ArrayList<String> result = new ArrayList<String>();
3      getPerms("", str, result);
4      return result;
5  }
6
7  void getPerms(String prefix, String remainder, ArrayList<String> result) {
8      if (remainder.length() == 0) result.add(prefix);
9
10     int len = remainder.length();
11     for (int i = 0; i < len; i++) {
12         String before = remainder.substring(0, i);
13         String after = remainder.substring(i + 1, len);
14         char c = remainder.charAt(i);
15         getPerms(prefix + c, before + after, result);
16     }
17 }

```

For a discussion of the runtime of this algorithm, see Example 12 on page 51.

8.8 Permutations with Duplicates: Write a method to compute all permutations of a string whose characters are not necessarily unique. The list of permutations should not have duplicates.

pg 135

SOLUTION

This is very similar to the previous problem, except that now we could potentially have duplicate characters in the word.

One simple way of handling this problem is to do the same work to check if a permutation has been created before and then, if not, add it to the list. A simple hash table will do the trick here. This solution will take $O(n!)$ time in the worst case (and, in fact, in all cases).

While it's true that we can't beat this worst case time, we should be able to design an algorithm to beat this in many cases. Consider a string with all duplicate characters, like aaaaaaaaaaaaaaaaa. This will take an extremely long time (since there are over 6 billion permutations of a 13-character string), even though there is only one unique permutation.

Ideally, we would like to only create the unique permutations, rather than creating every permutation and then ruling out the duplicates.

We can start with computing the count of each letter (easy enough to get this—just use a hash table). For a string such as aabbbbc, this would be:

a->2 | b->4 | c->1

Let's imagine generating a permutation of this string (now represented as a hash table). The first choice we make is whether to use an a, b, or c as the first character. After that, we have a subproblem to solve: find all permutations of the remaining characters, and append those to the already picked "prefix."

$$\begin{aligned}
 P(a \rightarrow 2 \mid b \rightarrow 4 \mid c \rightarrow 1) &= \{a + P(a \rightarrow 1 \mid b \rightarrow 4 \mid c \rightarrow 1)\} + \\
 &\quad \{b + P(a \rightarrow 2 \mid b \rightarrow 3 \mid c \rightarrow 1)\} + \\
 &\quad \{c + P(a \rightarrow 2 \mid b \rightarrow 4 \mid c \rightarrow 0)\} \\
 P(a \rightarrow 1 \mid b \rightarrow 4 \mid c \rightarrow 1) &= \{a + P(a \rightarrow 0 \mid b \rightarrow 4 \mid c \rightarrow 1)\} + \\
 &\quad \{b + P(a \rightarrow 1 \mid b \rightarrow 3 \mid c \rightarrow 1)\} + \\
 &\quad \{c + P(a \rightarrow 1 \mid b \rightarrow 4 \mid c \rightarrow 0)\} \\
 P(a \rightarrow 2 \mid b \rightarrow 3 \mid c \rightarrow 1) &= \{a + P(a \rightarrow 1 \mid b \rightarrow 3 \mid c \rightarrow 1)\} + \\
 &\quad \{b + P(a \rightarrow 2 \mid b \rightarrow 2 \mid c \rightarrow 1)\} + \\
 &\quad \{c + P(a \rightarrow 2 \mid b \rightarrow 3 \mid c \rightarrow 0)\} \\
 P(a \rightarrow 2 \mid b \rightarrow 4 \mid c \rightarrow 0) &= \{a + P(a \rightarrow 1 \mid b \rightarrow 4 \mid c \rightarrow 0)\} + \\
 &\quad \{b + P(a \rightarrow 2 \mid b \rightarrow 3 \mid c \rightarrow 0)\}
 \end{aligned}$$

Eventually, we'll get down to no more characters remaining.

The code below implements this algorithm.

```

1  ArrayList<String> printPerms(String s) {
2      ArrayList<String> result = new ArrayList<String>();
3      HashMap<Character, Integer> map = buildFreqTable(s);
4      printPerms(map, "", s.length(), result);
5      return result;
6  }
7
8  HashMap<Character, Integer> buildFreqTable(String s) {
9      HashMap<Character, Integer> map = new HashMap<Character, Integer>();
10     for (char c : s.toCharArray()) {
11         if (!map.containsKey(c)) {
12             map.put(c, 0);
13         }
14         map.put(c, map.get(c) + 1);
15     }
16     return map;
17 }
18
19 void printPerms(HashMap<Character, Integer> map, String prefix, int remaining,
20     ArrayList<String> result) {
21     /* Base case. Permutation has been completed. */
22     if (remaining == 0) {
23         result.add(prefix);
24         return;
25     }
26
27     /* Try remaining letters for next char, and generate remaining permutations. */

```

```

28     for (Character c : map.keySet()) {
29         int count = map.get(c);
30         if (count > 0) {
31             map.put(c, count - 1);
32             printPerms(map, prefix + c, remaining - 1, result);
33             map.put(c, count);
34         }
35     }
36 }

```

In situations where the string has many duplicates, this algorithm will run a lot faster than the earlier algorithm.

8.9 Pairs: Implement an algorithm to print all valid (i.e., properly opened and closed) combinations of n pairs of parentheses.

EXAMPLE

Input: 3

Output: ((())), (()()), (())(), ()(()), ()()()

pg 136

SOLUTION

Our first thought here might be to apply a recursive approach where we build the solution for $f(n)$ by adding pairs of parentheses to $f(n-1)$. That's certainly a good instinct.

Let's consider the solution for $n = 3$:

((())) (()()) (())() ()(()) ()()()

How might we build this from $n = 2$?

(()) ()()

We can do this by inserting a pair of parentheses inside every existing pair of parentheses, as well as one at the beginning of the string. Any other places that we could insert parentheses, such as at the end of the string, would reduce to the earlier cases.

So, we have the following:

```

(( )) -> (( ( )) /* inserted pair after 1st left paren */
      -> (( ( ( )) /* inserted pair after 2nd left paren */
      -> ( ( ( )) /* inserted pair at beginning of string */
(( ( )) -> (( ( ( )) /* inserted pair after 1st left paren */
      -> ( ( ( ( )) /* inserted pair after 2nd left paren */
      -> ( ( ( )) /* inserted pair at beginning of string */

```

But wait—we have some duplicate pairs listed. The string `()(())` is listed twice.

If we're going to apply this approach, we'll need to check for duplicate values before adding a string to our list.

```

1  Set<String> generateParens(int remaining) {
2      Set<String> set = new HashSet<String>();
3      if (remaining == 0) {
4          set.add("");
5      } else {
6          Set<String> prev = generateParens(remaining - 1);
7          for (String str : prev) {
8              for (int i = 0; i < str.length(); i++) {

```

```

9         if (str.charAt(i) == '(') {
10             String s = insertInside(str, i);
11             /* Add s to set if it's not already in there. Note: HashSet
12              * automatically checks for duplicates before adding, so an explicit
13              * check is not necessary. */
14             set.add(s);
15         }
16     }
17     set.add("(" + str);
18 }
19 }
20 return set;
21 }
22
23 String insertInside(String str, int leftIndex) {
24     String left = str.substring(0, leftIndex + 1);
25     String right = str.substring(leftIndex + 1, str.length());
26     return left + "(" + right;
27 }

```

This works, but it's not very efficient. We waste a lot of time coming up with the duplicate strings.

We can avoid this duplicate string issue by building the string from scratch. Under this approach, we add left and right parens, as long as our expression stays valid.

On each recursive call, we have the index for a particular character in the string. We need to select either a left or a right paren. When can we use a left paren, and when can we use a right paren?

1. *Left Paren:* As long as we haven't used up all the left parentheses, we can always insert a left paren.
2. *Right Paren:* We can insert a right paren as long as it won't lead to a syntax error. When will we get a syntax error? We will get a syntax error if there are more right parentheses than left.

So, we simply keep track of the number of left and right parentheses allowed. If there are left parens remaining, we'll insert a left paren and recurse. If there are more right parens remaining than left (i.e., if there are more left parens in use than right parens), then we'll insert a right paren and recurse.

```

1 void addParen(ArrayList<String> list, int leftRem, int rightRem, char[] str,
2               int index) {
3     if (leftRem < 0 || rightRem < leftRem) return; // invalid state
4
5     if (leftRem == 0 && rightRem == 0) { /* Out of left and right parentheses */
6         list.add(String.valueOf(str));
7     } else {
8         str[index] = '('; // Add left and recurse
9         addParen(list, leftRem - 1, rightRem, str, index + 1);
10
11         str[index] = ')'; // Add right and recurse
12         addParen(list, leftRem, rightRem - 1, str, index + 1);
13     }
14 }
15
16 ArrayList<String> generateParens(int count) {
17     char[] str = new char[count*2];
18     ArrayList<String> list = new ArrayList<String>();
19     addParen(list, count, count, str, 0);
20     return list;
21 }

```

Because we insert left and right parentheses at each index in the string, and we never repeat an index, each string is guaranteed to be unique.

8.10 Paint Fill: Implement the “paint fill” function that one might see on many image editing programs. That is, given a screen (represented by a two-dimensional array of colors), a point, and a new color, fill in the surrounding area until the color changes from the original color.

pg 136

SOLUTION

First, let’s visualize how this method works. When we call `paintFill` (i.e., “click” paint fill in the image editing application) on, say, a green pixel, we want to “bleed” outwards. Pixel by pixel, we expand outwards by calling `paintFill` on the surrounding pixel. When we hit a pixel that is not green, we stop.

We can implement this algorithm recursively:

```

1  enum Color { Black, White, Red, Yellow, Green }
2
3  boolean PaintFill(Color[][] screen, int r, int c, Color ncolor) {
4      if (screen[r][c] == ncolor) return false;
5      return PaintFill(screen, r, c, screen[r][c], ncolor);
6  }
7
8  boolean PaintFill(Color[][] screen, int r, int c, Color ocolor, Color ncolor) {
9      if (r < 0 || r >= screen.length || c < 0 || c >= screen[0].length) {
10         return false;
11     }
12
13     if (screen[r][c] == ocolor) {
14         screen[r][c] = ncolor;
15         PaintFill(screen, r - 1, c, ocolor, ncolor); // up
16         PaintFill(screen, r + 1, c, ocolor, ncolor); // down
17         PaintFill(screen, r, c - 1, ocolor, ncolor); // left
18         PaintFill(screen, r, c + 1, ocolor, ncolor); // right
19     }
20     return true;
21 }
```

If you used the variable names `x` and `y` to implement this, be careful about the ordering of the variables in `screen[y][x]`. Because `x` represents the *horizontal* axis (that is, it’s left to right), it actually corresponds to the column number, not the row number. The value of `y` equals the number of rows. This is a very easy place to make a mistake in an interview, as well as in your daily coding. It’s typically clearer to use `row` and `column` instead, as we’ve done here.

Does this algorithm seem familiar? It should! This is essentially depth-first search on a graph. At each pixel, we are searching outwards to each surrounding pixel. We stop once we’ve fully traversed all the surrounding pixels of this color.

We could alternatively implement this using breadth-first search.

8.11 Coins: Given an infinite number of quarters (25 cents), dimes (10 cents), nickels (5 cents), and pennies (1 cent), write code to calculate the number of ways of representing *n* cents.

pg 136

SOLUTION

This is a recursive problem, so let's figure out how to compute `makeChange(n)` using prior solutions (i.e., subproblems).

Let's say $n = 100$. We want to compute the number of ways of making change for 100 cents. What is the relationship between this problem and its subproblems?

We know that making change for 100 cents will involve either 0, 1, 2, 3, or 4 quarters. So:

```
makeChange(100) = makeChange(100 using 0 quarters) +
                  makeChange(100 using 1 quarter) +
                  makeChange(100 using 2 quarters) +
                  makeChange(100 using 3 quarters) +
                  makeChange(100 using 4 quarters)
```

Inspecting this further, we can see that some of these problems reduce. For example, `makeChange(100 using 1 quarter)` will equal `makeChange(75 using 0 quarters)`. This is because, if we must use exactly one quarter to make change for 100 cents, then our only remaining choices involve making change for the remaining 75 cents.

We can apply the same logic to `makeChange(100 using 2 quarters)`, `makeChange(100 using 3 quarters)` and `makeChange(100 using 4 quarters)`. We have thus reduced the above statement to the following.

```
makeChange(100) = makeChange(100 using 0 quarters) +
                  makeChange(75 using 0 quarters) +
                  makeChange(50 using 0 quarters) +
                  makeChange(25 using 0 quarters) +
                  1
```

Note that the final statement from above, `makeChange(100 using 4 quarters)`, equals 1. We call this "fully reduced."

Now what? We've used up all our quarters, so now we can start applying our next biggest denomination: dimes.

Our approach for quarters applies to dimes as well, but we apply this for *each* of the four of five parts of the above statement. So, for the first part, we get the following statements:

```
makeChange(100 using 0 quarters) = makeChange(100 using 0 quarters, 0 dimes) +
                                    makeChange(100 using 0 quarters, 1 dime) +
                                    makeChange(100 using 0 quarters, 2 dimes) +
                                    ...
                                    makeChange(100 using 0 quarters, 10 dimes)
```

```
makeChange(75 using 0 quarters) = makeChange(75 using 0 quarters, 0 dimes) +
                                    makeChange(75 using 0 quarters, 1 dime) +
                                    makeChange(75 using 0 quarters, 2 dimes) +
                                    ...
                                    makeChange(75 using 0 quarters, 7 dimes)
```

```
makeChange(50 using 0 quarters) = makeChange(50 using 0 quarters, 0 dimes) +
                                    makeChange(50 using 0 quarters, 1 dime) +
                                    makeChange(50 using 0 quarters, 2 dimes) +
```

```

...
makeChange(50 using 0 quarters, 5 dimes)

makeChange(25 using 0 quarters) = makeChange(25 using 0 quarters, 0 dimes) +
                                   makeChange(25 using 0 quarters, 1 dime) +
                                   makeChange(25 using 0 quarters, 2 dimes)

```

Each one of these, in turn, expands out once we start applying nickels. We end up with a tree-like recursive structure where each call expands out to four or more calls.

The base case of our recursion is the fully reduced statement. For example, `makeChange(50 using 0 quarters, 5 dimes)` is fully reduced to 1, since 5 dimes equals 50 cents.

This leads to a recursive algorithm that looks like this:

```

1  int makeChange(int amount, int[] denoms, int index) {
2      if (index >= denoms.length - 1) return 1; // last denom
3      int denomAmount = denoms[index];
4      int ways = 0;
5      for (int i = 0; i * denomAmount <= amount; i++) {
6          int amountRemaining = amount - i * denomAmount;
7          ways += makeChange(amountRemaining, denoms, index + 1);
8      }
9      return ways;
10 }
11
12 int makeChange(int n) {
13     int[] denoms = {25, 10, 5, 1};
14     return makeChange(n, denoms, 0);
15 }

```

This works, but it's not as optimal as it could be. The issue is that we will be recursively calling `makeChange` several times for the same values of `amount` and `index`.

We can resolve this issue by storing the previously computed values. We'll need to store a mapping from each pair (`amount`, `index`) to the precomputed result.

```

1  int makeChange(int n) {
2      int[] denoms = {25, 10, 5, 1};
3      int[][] map = new int[n + 1][denoms.length]; // precomputed vals
4      return makeChange(n, denoms, 0, map);
5  }
6
7  int makeChange(int amount, int[] denoms, int index, int[][] map) {
8      if (map[amount][index] > 0) { // retrieve value
9          return map[amount][index];
10     }
11     if (index >= denoms.length - 1) return 1; // one denom remaining
12     int denomAmount = denoms[index];
13     int ways = 0;
14     for (int i = 0; i * denomAmount <= amount; i++) {
15         // go to next denom, assuming i coins of denomAmount
16         int amountRemaining = amount - i * denomAmount;
17         ways += makeChange(amountRemaining, denoms, index + 1, map);
18     }
19     map[amount][index] = ways;
20     return ways;
21 }

```

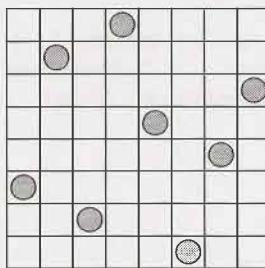
Note that we've used a two-dimensional array of integers to store the previously computed values. This is simpler, but takes up a little extra space. Alternatively, we could use an actual hash table that maps from amount to a new hash table, which then maps from denom to the precomputed value. There are other alternative data structures as well.

8.12 Eight Queens: Write an algorithm to print all ways of arranging eight queens on an 8x8 chess board so that none of them share the same row, column, or diagonal. In this case, "diagonal" means all diagonals, not just the two that bisect the board.

pg 136

SOLUTION

We have eight queens which must be lined up on an 8x8 chess board such that none share the same row, column or diagonal. So, we know that each row and column (and diagonal) must be used exactly once.



A "Solved" Board with 8 Queens

Picture the queen that is placed last, which we'll assume is on row 8. (This is an okay assumption to make since the ordering of placing the queens is irrelevant.) On which cell in row 8 is this queen? There are eight possibilities, one for each column.

So if we want to know all the valid ways of arranging 8 queens on an 8x8 chess board, it would be:

```
ways to arrange 8 queens on an 8x8 board =
  ways to arrange 8 queens on an 8x8 board with queen at (7, 0) +
  ways to arrange 8 queens on an 8x8 board with queen at (7, 1) +
  ways to arrange 8 queens on an 8x8 board with queen at (7, 2) +
  ways to arrange 8 queens on an 8x8 board with queen at (7, 3) +
  ways to arrange 8 queens on an 8x8 board with queen at (7, 4) +
  ways to arrange 8 queens on an 8x8 board with queen at (7, 5) +
  ways to arrange 8 queens on an 8x8 board with queen at (7, 6) +
  ways to arrange 8 queens on an 8x8 board with queen at (7, 7)
```

We can compute each one of these using a very similar approach:

```
ways to arrange 8 queens on an 8x8 board with queen at (7, 3) =
  ways to ... with queens at (7, 3) and (6, 0) +
  ways to ... with queens at (7, 3) and (6, 1) +
  ways to ... with queens at (7, 3) and (6, 2) +
  ways to ... with queens at (7, 3) and (6, 4) +
  ways to ... with queens at (7, 3) and (6, 5) +
  ways to ... with queens at (7, 3) and (6, 6) +
  ways to ... with queens at (7, 3) and (6, 7)
```

Note that we don't need to consider combinations with queens at (7, 3) and (6, 3), since this is a violation of the requirement that every queen is in its own row, column and diagonal.

Implementing this is now reasonably straightforward.

```

1  int GRID_SIZE = 8;
2
3  void placeQueens(int row, Integer[] columns, ArrayList<Integer[]> results) {
4      if (row == GRID_SIZE) { // Found valid placement
5          results.add(columns.clone());
6      } else {
7          for (int col = 0; col < GRID_SIZE; col++) {
8              if (checkValid(columns, row, col)) {
9                  columns[row] = col; // Place queen
10                 placeQueens(row + 1, columns, results);
11             }
12         }
13     }
14 }
15
16 /* Check if (row1, column1) is a valid spot for a queen by checking if there is a
17 * queen in the same column or diagonal. We don't need to check it for queens in
18 * the same row because the calling placeQueen only attempts to place one queen at
19 * a time. We know this row is empty. */
20 boolean checkValid(Integer[] columns, int row1, int column1) {
21     for (int row2 = 0; row2 < row1; row2++) {
22         int column2 = columns[row2];
23         /* Check if (row2, column2) invalidates (row1, column1) as a
24          * queen spot. */
25
26         /* Check if rows have a queen in the same column */
27         if (column1 == column2) {
28             return false;
29         }
30
31         /* Check diagonals: if the distance between the columns equals the distance
32          * between the rows, then they're in the same diagonal. */
33         int columnDistance = Math.abs(column2 - column1);
34
35         /* row1 > row2, so no need for abs */
36         int rowDistance = row1 - row2;
37         if (columnDistance == rowDistance) {
38             return false;
39         }
40     }
41     return true;
42 }

```

Observe that since each row can only have one queen, we don't need to store our board as a full 8x8 matrix. We only need a single array where `column[r] = c` indicates that row `r` has a queen at column `c`.

8.13 Stack of Boxes: You have a stack of n boxes, with widths w_i , heights h_i , and depths d_i . The boxes cannot be rotated and can only be stacked on top of one another if each box in the stack is strictly larger than the box above it in width, height, and depth. Implement a method to compute the height of the tallest possible stack. The height of a stack is the sum of the heights of each box.

pg 136

SOLUTION

To tackle this problem, we need to recognize the relationship between the different subproblems.

Solution #1

Imagine we had the following boxes: $b_1, b_2, \dots, b_n$. The biggest stack that we can build with all the boxes equals the max of (biggest stack with bottom b_1 , biggest stack with bottom $b_2, \dots$, biggest stack with bottom b_n). That is, if we experimented with each box as a bottom and built the biggest stack possible with each, we would find the biggest stack possible.

But, how would we find the biggest stack with a particular bottom? Essentially the same way. We experiment with different boxes for the second level, and so on for each level.

Of course, we only experiment with valid boxes. If b_5 is bigger than b_1 , then there's no point in trying to build a stack that looks like $\{b_1, b_5, \dots\}$. We already know b_1 can't be below b_5 .

We can perform a small optimization here. The requirements of this problem stipulate that the lower boxes must be strictly greater than the higher boxes in all dimensions. Therefore, if we sort (descending order) the boxes on a dimension—any dimension—then we know we don't have to look backwards in the list. The box b_1 cannot be on top of box b_5 , since its height (or whatever dimension we sorted on) is greater than b_5 's height.

The code below implements this algorithm recursively.

```

1  int createStack(ArrayList<Box> boxes) {
2      /* Sort in descending order by height. */
3      Collections.sort(boxes, new BoxComparator());
4      int maxHeight = 0;
5      for (int i = 0; i < boxes.size(); i++) {
6          int height = createStack(boxes, i);
7          maxHeight = Math.max(maxHeight, height);
8      }
9      return maxHeight;
10 }
11
12 int createStack(ArrayList<Box> boxes, int bottomIndex) {
13     Box bottom = boxes.get(bottomIndex);
14     int maxHeight = 0;
15     for (int i = bottomIndex + 1; i < boxes.size(); i++) {
16         if (boxes.get(i).canBeAbove(bottom)) {
17             int height = createStack(boxes, i);
18             maxHeight = Math.max(height, maxHeight);
19         }
20     }
21     maxHeight += bottom.height;
22     return maxHeight;
23 }
24
25 class BoxComparator implements Comparator<Box> {

```

```

26  @Override
27  public int compare(Box x, Box y){
28      return y.height - x.height;
29  }
30 }

```

The problem in this code is that it gets very inefficient. We try to find the best solution that looks like $\{b_3, b_4, \dots\}$ even though we may have already found the best solution with b_4 at the bottom. Instead of generating these solutions from scratch, we can cache these results using memoization.

```

1  int createStack(ArrayList<Box> boxes) {
2      Collections.sort(boxes, new BoxComparator());
3      int maxHeight = 0;
4      int[] stackMap = new int[boxes.size()];
5      for (int i = 0; i < boxes.size(); i++) {
6          int height = createStack(boxes, i, stackMap);
7          maxHeight = Math.max(maxHeight, height);
8      }
9      return maxHeight;
10 }
11
12 int createStack(ArrayList<Box> boxes, int bottomIndex, int[] stackMap) {
13     if (bottomIndex < boxes.size() && stackMap[bottomIndex] > 0) {
14         return stackMap[bottomIndex];
15     }
16
17     Box bottom = boxes.get(bottomIndex);
18     int maxHeight = 0;
19     for (int i = bottomIndex + 1; i < boxes.size(); i++) {
20         if (boxes.get(i).canBeAbove(bottom)) {
21             int height = createStack(boxes, i, stackMap);
22             maxHeight = Math.max(height, maxHeight);
23         }
24     }
25     maxHeight += bottom.height;
26     stackMap[bottomIndex] = maxHeight;
27     return maxHeight;
28 }

```

Because we're only mapping from an index to a height, we can just use an integer array for our "hash table."

Be very careful here with what each spot in the hash table represents. In this code, `stackMap[i]` represents the tallest stack with box `i` at the bottom. Before pulling the value from the hash table, you have to ensure that box `i` can be placed on top of the current bottom.

It helps to keep the line that recalls from the hash table symmetric with the one that inserts. For example, in this code, we recall from the hash table with `bottomIndex` at the start of the method. We insert into the hash table with `bottomIndex` at the end.

Solution #2

Alternatively, we can think about the recursive algorithm as making a choice, at each step, whether to put a particular box in the stack. (We will again sort our boxes in descending order by a dimension, such as height.)

First, we choose whether or not to put box 0 in the stack. Take one recursive path with box 0 at the bottom and one recursive path without box 0. Return the better of the two options.

Then, we choose whether or not to put box 1 in the stack. Take one recursive path with box 1 at the bottom and one path without box 1. Return the better of the two options.

We will again use memoization to cache the height of the tallest stack with a particular bottom.

```

1  int createStack(ArrayList<Box> boxes) {
2      Collections.sort(boxes, new BoxComparator());
3      int[] stackMap = new int[boxes.size()];
4      return createStack(boxes, null, 0, stackMap);
5  }
6
7  int createStack(ArrayList<Box> boxes, Box bottom, int offset, int[] stackMap) {
8      if (offset >= boxes.size()) return 0; // Base case
9
10     /*height with this bottom */
11     Box newBottom = boxes.get(offset);
12     int heightWithBottom = 0;
13     if (bottom == null || newBottom.canBeAbove(bottom)) {
14         if (stackMap[offset] == 0) {
15             stackMap[offset] = createStack(boxes, newBottom, offset + 1, stackMap);
16             stackMap[offset] += newBottom.height;
17         }
18         heightWithBottom = stackMap[offset];
19     }
20
21     /*without this bottom */
22     int heightWithoutBottom = createStack(boxes, bottom, offset + 1, stackMap);
23
24     /* Return better of two options. */
25     return Math.max(heightWithBottom, heightWithoutBottom);
26 }

```

Again, pay close attention to when you recall and insert values into the hash table. It's typically best if these are symmetric, as they are in lines 15 and 16-18.

8.14 Boolean Evaluation: Given a boolean expression consisting of the symbols 0 (false), 1 (true), & (AND), | (OR), and ^ (XOR), and a desired boolean result value `result`, implement a function to count the number of ways of parenthesizing the expression such that it evaluates to `result`. The expressions should be fully parenthesized (e.g., $(0)^{(1)}$) but not extraneously (e.g., $((0))^{(1)}$).

EXAMPLE

```

countEval("1^0|0|1", false) -> 2
countEval("0&0&0&1^1|0", true) -> 10

```

pg 136

SOLUTION

As in other recursive problems, the key to this problem is to figure out the relationship between a problem and its subproblems.

Brute Force

Consider an expression like $0^00^1|1$ and the target result `true`. How can we break down `countEval("0^00^1|1", true)` into smaller problems?

We could just essentially iterate through each possible place to put a parenthesis.

```
countEval("0^0&0^1|1", true) =
    countEval("0^0&0^1|1" where paren around char 1, true)
+ countEval("0^0&0^1|1" where paren around char 3, true)
+ countEval("0^0&0^1|1" where paren around char 5, true)
+ countEval("0^0&0^1|1" where paren around char 7, true)
```

Now what? Let's look at just one of those expressions—the paren around char 3. This gives us $(0^00) \& (0^11)$.

In order to make that expression true, both the left and right sides must be true. So:

```
left = "0^0"
right = "0^1|1"
countEval(left & right, true) = countEval(left, true) * countEval(right, true)
```

The reason we multiply the results of the left and right sides is that each result from the two sides can be paired up with each other to form a unique combination.

Each of those terms can now be decomposed into smaller problems in a similar process.

What happens when we have an `"|"` (OR)? Or an `"^"` (XOR)?

If it's an OR, then either the left or the right side must be true—or both.

```
countEval(left | right, true) = countEval(left, true) * countEval(right, false)
+ countEval(left, false) * countEval(right, true)
+ countEval(left, true) * countEval(right, true)
```

If it's an XOR, then the left or the right side can be true, but not both.

```
countEval(left ^ right, true) = countEval(left, true) * countEval(right, false)
+ countEval(left, false) * countEval(right, true)
```

What if we were trying to make the result false instead? We can switch up the logic from above:

```
countEval(left & right, false) = countEval(left, true) * countEval(right, false)
+ countEval(left, false) * countEval(right, true)
+ countEval(left, false) * countEval(right, false)
countEval(left | right, false) = countEval(left, false) * countEval(right, false)
countEval(left ^ right, false) = countEval(left, false) * countEval(right, false)
+ countEval(left, true) * countEval(right, true)
```

Alternatively, we can just use the same logic from above and subtract it out from the total number of ways of evaluating the expression.

```
totalEval(left) = countEval(left, true) + countEval(left, false)
totalEval(right) = countEval(right, true) + countEval(right, false)
totalEval(expression) = totalEval(left) * totalEval(right)
countEval(expression, false) = totalEval(expression) - countEval(expression, true)
```

This makes the code a bit more concise.

```
1 int countEval(String s, boolean result) {
2     if (s.length() == 0) return 0;
3     if (s.length() == 1) return stringToBool(s) == result ? 1 : 0;
4
5     int ways = 0;
6     for (int i = 1; i < s.length(); i += 2) {
7         char c = s.charAt(i);
8         String left = s.substring(0, i);
9         String right = s.substring(i + 1, s.length());
10
11         /* Evaluate each side for each result. */
12         int leftTrue = countEval(left, true);
13         int leftFalse = countEval(left, false);
14         int rightTrue = countEval(right, true);
```

```

15     int rightFalse = countEval(right, false);
16     int total = (leftTrue + leftFalse) * (rightTrue + rightFalse);
17
18     int totalTrue = 0;
19     if (c == '^') { // required: one true and one false
20         totalTrue = leftTrue * rightFalse + leftFalse * rightTrue;
21     } else if (c == '&') { // required: both true
22         totalTrue = leftTrue * rightTrue;
23     } else if (c == '|') { // required: anything but both false
24         totalTrue = leftTrue * rightTrue + leftFalse * rightTrue +
25                     leftTrue * rightFalse;
26     }
27
28     int subWays = result ? totalTrue : total - totalTrue;
29     ways += subWays;
30 }
31
32 return ways;
33 }
34
35 boolean stringToBool(String c) {
36     return c.equals("1") ? true : false;
37 }

```

Note that the tradeoff of computing the false results from the true ones, and of computing the {leftTrue, rightTrue, leftFalse, and rightFalse} values upfront, is a small amount of extra work in some cases. For example, if we're looking for the ways that an AND (&) can result in true, we never would have needed the leftFalse and rightFalse results. Likewise, if we're looking for the ways that an OR (|) can result in false, we never would have needed the leftTrue and rightTrue results.

Our current code is blind to what we do and don't actually need to do and instead just computes all of the values. This is probably a reasonable tradeoff to make (especially given the constraints of whiteboard coding) as it makes our code substantially shorter and less tedious to write. Whichever approach you make, you should discuss the tradeoffs with your interviewer.

That said, there are more important optimizations we can make.

Optimized Solutions

If we follow the recursive path, we'll note that we end up doing the same computation repeatedly.

Consider the expression $0^00&0^1|1$ and these recursion paths:

- Add parens around char 1. $(0)^{(0&0^1|1)}$
 - » Add parens around char 3. $(0)^{((0)&(0^1|1))}$
- Add parens around char 3. $(0^00)&(0^1|1)$
 - » Add parens around char 1. $((0)^{(0)})&(0^1|1)$

Although these two expressions are different, they have a similar component: $(0^1|1)$. We should reuse our effort on this.

We can do this by using memoization, or a hash table. We just need to store the result of `countEval(expression, result)` for each expression and result. If we see an expression that we've calculated before, we just return it from the cache.

```

1  int countEval(String s, boolean result, HashMap<String, Integer> memo) {
2      if (s.length() == 0) return 0;
3      if (s.length() == 1) return stringToBool(s) == result ? 1 : 0;

```

```

4     if (memo.containsKey(result + s)) return memo.get(result + s);
5
6     int ways = 0;
7
8     for (int i = 1; i < s.length(); i += 2) {
9         char c = s.charAt(i);
10        String left = s.substring(0, i);
11        String right = s.substring(i + 1, s.length());
12        int leftTrue = countEval(left, true, memo);
13        int leftFalse = countEval(left, false, memo);
14        int rightTrue = countEval(right, true, memo);
15        int rightFalse = countEval(right, false, memo);
16        int total = (leftTrue + leftFalse) * (rightTrue + rightFalse);
17
18        int totalTrue = 0;
19        if (c == '^') {
20            totalTrue = leftTrue * rightFalse + leftFalse * rightTrue;
21        } else if (c == '&') {
22            totalTrue = leftTrue * rightTrue;
23        } else if (c == '|') {
24            totalTrue = leftTrue * rightTrue + leftFalse * rightTrue +
25                leftTrue * rightFalse;
26        }
27
28        int subWays = result ? totalTrue : total - totalTrue;
29        ways += subWays;
30    }
31
32    memo.put(result + s, ways);
33    return ways;
34 }

```

The added benefit of this is that we could actually end up with the same substring in multiple parts of the expression. For example, an expression like $0^1^0 \& 0^1^0$ has two instances of 0^1^0 . By caching the result of the substring value in a memoization table, we'll get to reuse the result for the right part of the expression after computing it for the left.

There is one further optimization we can make, but it's far beyond the scope of the interview. There is a closed form expression for the number of ways of parenthesizing an expression, but you wouldn't be expected to know it. It is given by the Catalan numbers, where n is the number of operators:

$$C_n = \frac{(2n)!}{(n+1)!n!}$$

We could use this to compute the total ways of evaluating the expression. Then, rather than computing `leftTrue` and `leftFalse`, we just compute one of those and calculate the other using the Catalan numbers. We would do the same thing for the right side.

9

Solutions to System Design and Scalability

9.1 Stock Data: Imagine you are building some sort of service that will be called by up to 1,000 client applications to get simple end-of-day stock price information (open, close, high, low). You may assume that you already have the data, and you can store it in any format you wish. How would you design the client-facing service that provides the information to client applications? You are responsible for the development, rollout, and ongoing monitoring and maintenance of the feed. Describe the different methods you considered and why you would recommend your approach. Your service can use any technologies you wish, and can distribute the information to the client applications in any mechanism you choose.

pg 144

SOLUTION

From the statement of the problem, we want to focus on how we actually distribute the information to clients. We can assume that we have some scripts that magically collect the information.

We want to start off by thinking about what the different aspects we should consider in a given proposal are:

- *Client Ease of Use:* We want the service to be easy for the clients to implement and useful for them.
- *Ease for Ourselves:* This service should be as easy as possible for us to implement, as we shouldn't impose unnecessary work on ourselves. We need to consider in this not only the cost of implementing, but also the cost of maintenance.
- *Flexibility for Future Demands:* This problem is stated in a "what would you do in the real world" way, so we should think like we would in a real-world problem. Ideally, we do not want to overly constrain ourselves in the implementation, such that we can't be flexible if the requirements or demands change.
- *Scalability and Efficiency:* We should be mindful of the efficiency of our solution, so as not to overly burden our service.

With this framework in mind, we can consider various proposals.

Proposal #1

One option is that we could keep the data in simple text files and let clients download the data through some sort of FTP server. This would be easy to maintain in some sense, since files can be easily viewed and backed up, but it would require more complex parsing to do any sort of query. And, if additional data were added to our text file, it might break the clients' parsing mechanism.

Proposal #2

We could use a standard SQL database, and let the clients plug directly into that. This would provide the following benefits:

- Facilitates an easy way for the clients to do query processing over the data, in case there are additional features we need to support. For example, we could easily and efficiently perform a query such as “return all stocks having an open price greater than N and a closing price less than M.”
- Rolling back, backing up data, and security could be provided using standard database features. We don’t have to “reinvent the wheel,” so it’s easy for us to implement.
- Reasonably easy for the clients to integrate into existing applications. SQL integration is a standard feature in software development environments.

What are the disadvantages of using a SQL database?

- It’s much heavier weight than we really need. We don’t necessarily need all the complexity of a SQL backend to support a feed of a few bits of information.
- It’s difficult for humans to be able to read it, so we’ll likely need to implement an additional layer to view and maintain the data. This increases our implementation costs.
- Security: While a SQL database offers pretty well defined security levels, we would still need to be very careful to not give clients access that they shouldn’t have. Additionally, even if clients aren’t doing anything “malicious,” they might perform expensive and inefficient queries, and our servers would bear the costs of that.

These disadvantages don’t mean that we shouldn’t provide SQL access. Rather, they mean that we should be aware of the disadvantages.

Proposal #3

XML is another great option for distributing the information. Our data has fixed format and fixed size: company_name, open, high, low, closing price. The XML could look like this:

```

1  <root>
2    <date value="2008-10-12">
3      <company name="foo">
4        <open>126.23</open>
5        <high>130.27</high>
6        <low>122.83</low>
7        <closingPrice>127.30</closingPrice>
8      </company>
9      <company name="bar">
10       <open>52.73</open>
11       <high>60.27</high>
12       <low>50.29</low>
13       <closingPrice>54.91</closingPrice>
14     </company>
15   </date>
16   <date value="2008-10-11"> . . . </date>
17 </root>

```

The advantages of this approach include the following:

- It’s very easy to distribute, and it can also be easily read by both machines and humans. This is one reason that XML is a standard data model to share and distribute data.
- Most languages have a library to perform XML parsing, so it’s reasonably easy for clients to implement.

- We can add new data to the XML file by adding additional nodes. This would not break the client's parser (provided they have implemented their parser in a reasonable way).
- Since the data is being stored as XML files, we can use existing tools for backing up the data. We don't need to implement our own backup tool.

The disadvantages may include:

- This solution sends the clients all the information, even if they only want part of it. It is inefficient in that way.
- Performing any queries on the data requires parsing the entire file.

Regardless of which solution we use for data storage, we could provide a web service (e.g., SOAP) for client data access. This adds a layer to our work, but it can provide additional security, and it may even make it easier for clients to integrate the system.

However—and this is a pro and a con—clients will be limited to grabbing the data only how we expect or want them to. By contrast, in a pure SQL implementation, clients could query for the highest stock price, even if this wasn't a procedure we "expected" them to need.

So which one of these would we use? There's no clear answer. The pure text file solution is probably a bad choice, but you can make a compelling argument for the SQL or XML solution, with or without a web service.

The goal of a question like this is not to see if you get the "correct" answer (there is no single correct answer). Rather, it's to see how you design a system, and how you evaluate trade-offs.

9.2 Social Network: How would you design the data structures for a very large social network like Facebook or LinkedIn? Describe how you would design an algorithm to show the shortest path between two people (e.g., Me -> Bob -> Susan -> Jason -> You).

pg 145

SOLUTION

A good way to approach this problem is to remove some of the constraints and solve it for that situation first.

Step 1: Simplify the Problem—Forget About the Millions of Users

First, let's forget that we're dealing with millions of users. Design this for the simple case.

We can construct a graph by treating each person as a node and letting an edge between two nodes indicate that the two users are friends.

If I wanted to find the path between two people, I could start with one person and do a simple breadth-first search.

Why wouldn't a depth-first search work well? First, depth-first search would just find a path. It wouldn't necessarily find the shortest path. Second, even if we just needed any path, it would be very inefficient. Two users might be only one degree of separation apart, but I could search millions of nodes in their "subtrees" before finding this relatively immediate connection.

Alternatively, I could do what's called a bidirectional breadth-first search. This means doing two breadth-first searches, one from the source and one from the destination. When the searches collide, we know we've found a path.

In the implementation, we'll use two classes to help us. `BFSData` holds the data we need for a breadth-first search, such as the `isVisited` hash table and the `toVisit` queue. `PathNode` will represent the path as we're searching it, storing each `Person` and the `previousNode` we visited in this path.

```

1  LinkedList<Person> findPathBiBFS(HashMap<Integer, Person> people, int source,
2                                     int destination) {
3      BFSData sourceData = new BFSData(people.get(source));
4      BFSData destData = new BFSData(people.get(destination));
5
6      while (!sourceData.isFinished() && !destData.isFinished()) {
7          /*Search out from source. */
8          Person collision = searchLevel(people, sourceData, destData);
9          if (collision != null) {
10             return mergePaths(sourceData, destData, collision.getID());
11         }
12
13         /*Search out from destination. */
14         collision = searchLevel(people, destData, sourceData);
15         if (collision != null) {
16             return mergePaths(sourceData, destData, collision.getID());
17         }
18     }
19     return null;
20 }
21
22 /*Search one level and return collision, if any. */
23 Person searchLevel(HashMap<Integer, Person> people, BFSData primary,
24                   BFSData secondary) {
25     /*We only want to search one level at a time. Count how many nodes are
26     * currently in the primary's level and only do that many nodes. We'll continue
27     * to add nodes to the end. */
28     int count = primary.toVisit.size();
29     for (int i = 0; i < count; i++) {
30         /*Pull out first node. */
31         PathNode pathNode = primary.toVisit.poll();
32         int personId = pathNode.getPerson().getID();
33
34         /*Check if it's already been visited. */
35         if (secondary.visited.containsKey(personId)) {
36             return pathNode.getPerson();
37         }
38
39         /*Add friends to queue. */
40         Person person = pathNode.getPerson();
41         ArrayList<Integer> friends = person.getFriends();
42         for (int friendId : friends) {
43             if (!primary.visited.containsKey(friendId)) {
44                 Person friend = people.get(friendId);
45                 PathNode next = new PathNode(friend, pathNode);
46                 primary.visited.put(friendId, next);
47                 primary.toVisit.add(next);
48             }
49         }
50     }
51     return null;
52 }
53

```

```

54  /* Merge paths where searches met at connection. */
55  LinkedList<Person> mergePaths(BFSData bfs1, BFSData bfs2, int connection) {
56      PathNode end1 = bfs1.visited.get(connection); // end1 -> source
57      PathNode end2 = bfs2.visited.get(connection); // end2 -> dest
58      LinkedList<Person> pathOne = end1.collapse(false);
59      LinkedList<Person> pathTwo = end2.collapse(true); // reverse
60      pathTwo.removeFirst(); // remove connection
61      pathOne.addAll(pathTwo); // add second path
62      return pathOne;
63  }
64
65  class PathNode {
66      private Person person = null;
67      private PathNode previousNode = null;
68      public PathNode(Person p, PathNode previous) {
69          person = p;
70          previousNode = previous;
71      }
72
73      public Person getPerson() { return person; }
74
75      public LinkedList<Person> collapse(boolean startsWithRoot) {
76          LinkedList<Person> path = new LinkedList<Person>();
77          PathNode node = this;
78          while (node != null) {
79              if (startsWithRoot) {
80                  path.addLast(node.person);
81              } else {
82                  path.addFirst(node.person);
83              }
84              node = node.previousNode;
85          }
86          return path;
87      }
88  }
89
90  class BFSData {
91      public Queue<PathNode> toVisit = new LinkedList<PathNode>();
92      public HashMap<Integer, PathNode> visited =
93          new HashMap<Integer, PathNode>();
94
95      public BFSData(Person root) {
96          PathNode sourcePath = new PathNode(root, null);
97          toVisit.add(sourcePath);
98          visited.put(root.getID(), sourcePath);
99      }
100
101      public boolean isFinished() {
102          return toVisit.isEmpty();
103      }
104  }

```

Many people are surprised that this is faster. Some quick math can explain why.

Suppose every person has k friends, and node S and node D have a friend C in common.

- Traditional breadth-first search from S to D : We go through roughly $k+k*k$ nodes: each of S 's k friends, and then each of their k friends.

- Bidirectional breadth-first search: We go through $2k$ nodes: each of S 's k friends and each of D 's k friends.

Of course, $2k$ is much less than $k+k*k$.

Generalizing this to a path of length q , we have this:

- BFS: $O(k^q)$
- Bidirectional BFS: $O(k^{q/2} + k^{q/2})$, which is just $O(k^{q/2})$

If you imagine a path like $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$ where each person has 100 friends, this is a big difference. BFS will require looking at 100 million (100^4) nodes. A bidirectional BFS will require looking at only 20,000 nodes (2×100^2).

A bidirectional BFS will generally be faster than the traditional BFS. However, it requires actually having access to both the source node and the destination nodes, which is not always the case.

Step 2: Handle the Millions of Users

When we deal with a service the size of LinkedIn or Facebook, we cannot possibly keep all of our data on one machine. That means that our simple Person data structure from above doesn't quite work—our friends may not live on the same machine as we do. Instead, we can replace our list of friends with a list of their IDs, and traverse as follows:

1. For each friend ID: `int machine_index = getMachineIDForUser(personID);`
2. Go to machine `#machine_index`
3. On that machine, do: `Person friend = getPersonWithID(person_id);`

The code below outlines this process. We've defined a class `Server`, which holds a list of all the machines, and a class `Machine`, which represents a single machine. Both classes have hash tables to efficiently lookup data.

```

1  class Server {
2      HashMap<Integer, Machine> machines = new HashMap<Integer, Machine>();
3      HashMap<Integer, Integer> personToMachineMap = new HashMap<Integer, Integer>();
4
5      public Machine getMachineWithId(int machineID) {
6          return machines.get(machineID);
7      }
8
9      public int getMachineIDForUser(int personID) {
10         Integer machineID = personToMachineMap.get(personID);
11         return machineID == null ? -1 : machineID;
12     }
13
14     public Person getPersonWithID(int personID) {
15         Integer machineID = personToMachineMap.get(personID);
16         if (machineID == null) return null;
17
18         Machine machine = getMachineWithId(machineID);
19         if (machine == null) return null;
20
21         return machine.getPersonWithID(personID);
22     }
23 }
24
25 class Person {

```

```
26 private ArrayList<Integer> friends = new ArrayList<Integer>();
27 private int personID;
28 private String info;
29
30 public Person(int id) { this.personID = id; }
31 public String getInfo() { return info; }
32 public void setInfo(String info) { this.info = info; }
33 public ArrayList<Integer> getFriends() { return friends; }
34 public int getID() { return personID; }
35 public void addFriend(int id) { friends.add(id); }
36 }
```

There are more optimizations and follow-up questions here than we could possibly discuss, but here are just a few possibilities.

Optimization: Reduce machine jumps

Jumping from one machine to another is expensive. Instead of randomly jumping from machine to machine with each friend, try to batch these jumps—e.g., if five of my friends live on one machine, I should look them up all at once.

Optimization: Smart division of people and machines

People are much more likely to be friends with people who live in the same country as they do. Rather than randomly dividing people across machines, try to divide them by country, city, state, and so on. This will reduce the number of jumps.

Question: Breadth-first search usually requires “marking” a node as visited. How do you do that in this case?

Usually, in BFS, we mark a node as visited by setting a `visited` flag in its node class. Here, we don't want to do that. There could be multiple searches going on at the same time, so it's a bad idea to just edit our data.

Instead, we could mimic the marking of nodes with a hash table to look up a node id and determine whether it's been visited.

Other Follow-Up Questions:

- In the real world, servers fail. How does this affect you?
- How could you take advantage of caching?
- Do you search until the end of the graph (infinite)? How do you decide when to give up?
- In real life, some people have more friends of friends than others, and are therefore more likely to make a path between you and someone else. How could you use this data to pick where to start traversing?

These are just a few of the follow-up questions you or the interviewer could raise. There are many others.

9.3 Web Crawler: If you were designing a web crawler, how would you avoid getting into infinite loops?

pg 145

SOLUTION

The first thing to ask ourselves in this problem is how an infinite loop might occur. The simplest answer is that, if we picture the web as a graph of links, an infinite loop will occur when a cycle occurs.

To prevent infinite loops, we just need to detect cycles. One way to do this is to create a hash table where we set `hash[v]` to `true` after we visit page `v`.

We can crawl the web using breadth-first search. Each time we visit a page, we gather all its links and insert them at the end of a queue. If we've already visited a page, we ignore it.

This is great—but what does it mean to visit page `v`? Is page `v` defined based on its content or its URL?

If it's defined based on its URL, we must recognize that URL parameters might indicate a completely different page. For example, the page `www.careercup.com/page?pid=microsoft-interview-questions` is totally different from the page `www.careercup.com/page?pid=google-interview-questions`. But, we can also append URL parameters arbitrarily to any URL without truly changing the page, provided it's not a parameter that the web application recognizes and handles. The page `www.careercup.com?foobar=hello` is the same as `www.careercup.com`.

"Okay, then," you might say, "let's define it based on its content." That sounds good too, at first, but it also doesn't quite work. Suppose I have some randomly generated content on the `careercup.com` home page. Is it a different page each time you visit it? Not really.

The reality is that there is probably no perfect way to define a "different" page, and this is where this problem gets tricky.

One way to tackle this is to have some sort of estimation for degree of similarity. If, based on the content and the URL, a page is deemed to be sufficiently similar to other pages, we *deprioritize* crawling its children. For each page, we would come up with some sort of signature based on snippets of the content and the page's URL.

Let's see how this would work.

We have a database which stores a list of items we need to crawl. On each iteration, we select the highest priority page to crawl. We then do the following:

1. Open up the page and create a signature of the page based on specific subsections of the page and its URL.
2. Query the database to see whether anything with this signature has been crawled recently.
3. If something with this signature has been recently crawled, insert this page back into the database at a low priority.
4. If not, crawl the page and insert its links into the database.

Under the above implementation, we never "complete" crawling the web, but we will avoid getting stuck in a loop of pages. If we want to allow for the possibility of "finishing" crawling the web (which would clearly happen only if the "web" were actually a smaller system, like an intranet), then we can set a minimum priority that a page must have to be crawled.

This is just one, simplistic solution, and there are many others that are equally valid. A problem like this will more likely resemble a conversation with your interviewer which could take any number of paths. In fact, the discussion of this problem could have taken the path of the very next problem.

9.4 Duplicate URLs: You have 10 billion URLs. How do you detect the duplicate documents? In this case, assume “duplicate” means that the URLs are identical.

pg 145

SOLUTION

Just how much space do 10 billion URLs take up? If each URL is an average of 100 characters, and each character is 4 bytes, then this list of 10 billion URLs will take up about 4 terabytes. We are probably not going to hold that much data in memory.

But, let’s just pretend for a moment that we were miraculously holding this data in memory, since it’s useful to first construct a solution for the simple version. Under this version of the problem, we would just create a hash table where each URL maps to `true` if it’s already been found elsewhere in the list. (As an alternative solution, we could sort the list and look for the duplicate values that way. That will take a bunch of extra time and offers few advantages.)

Now that we have a solution for the simple version, what happens when we have all 4000 gigabytes of data and we can’t store it all in memory? We could solve this either by storing some of the data on disk or by splitting up the data across machines.

Solution #1: Disk Storage

If we stored all the data on one machine, we would do two passes of the document. The first pass would split the list of URLs into 4000 chunks of 1 GB each. An easy way to do that might be to store each URL `u` in a file named `<x>.txt` where $x = \text{hash}(u) \% 4000$. That is, we divide up the URLs based on their hash value (modulo the number of chunks). This way, all URLs with the same hash value would be in the same file.

In the second pass, we would essentially implement the simple solution we came up with earlier: load each file into memory, create a hash table of the URLs, and look for duplicates.

Solution #2: Multiple Machines

The other solution is to perform essentially the same procedure, but to use multiple machines. In this solution, rather than storing the data in file `<x>.txt`, we would send the URL to machine `x`.

Using multiple machines has pros and cons.

The main pro is that we can parallelize the operation, such that all 4000 chunks are processed simultaneously. For large amounts of data, this might result in a faster solution.

The disadvantage though is that we are now relying on 4000 different machines to operate perfectly. That may not be realistic (particularly with more data and more machines), and we’ll need to start considering how to handle failure. Additionally, we have increased the complexity of the system simply by involving so many machines.

Both are good solutions, though, and both should be discussed with your interviewer.

9.5 Cache: Imagine a web server for a simplified search engine. This system has 100 machines to respond to search queries, which may then call out using `processSearch(string query)` to another cluster of machines to actually get the result. The machine which responds to a given query is chosen at random, so you cannot guarantee that the same machine will always respond to the same request. The method `processSearch` is very expensive. Design a caching mechanism to cache the results of the most recent queries. Be sure to explain how you would update the cache when data changes.

pg 145

SOLUTION

Before getting into the design of this system, we first have to understand what the question means. Many of the details are somewhat ambiguous, as is expected in questions like this. We will make reasonable assumptions for the purposes of this solution, but you should discuss these details—in depth—with your interviewer.

Assumptions

Here are a few of the assumptions we make for this solution. Depending on the design of your system and how you approach the problem, you may make other assumptions. Remember that while some approaches are better than others, there is no one “correct” approach.

- Other than calling out to `processSearch` as necessary, all query processing happens on the initial machine that was called.
- The number of queries we wish to cache is large (millions).
- Calling between machines is relatively quick.
- The result for a given query is an ordered list of URLs, each of which has an associated 50 character title and 200 character summary.
- The most popular queries are extremely popular, such that they would always appear in the cache.

Again, these aren’t the *only* valid assumptions. This is just one reasonable set of assumptions.

System Requirements

When designing the cache, we know we’ll need to support two primary functions:

- Efficient lookups given a key.
- Expiration of old data so that it can be replaced with new data.

In addition, we must also handle updating or clearing the cache when the results for a query change. Because some queries are very common and may permanently reside in the cache, we cannot just wait for the cache to naturally expire.

Step 1: Design a Cache for a Single System

A good way to approach this problem is to start by designing it for a single machine. So, how would you create a data structure that enables you to easily purge old data and also efficiently look up a value based on a key?

- A linked list would allow easy purging of old data, by moving “fresh” items to the front. We could implement it to remove the last element of the linked list when the list exceeds a certain size.

- A hash table allows efficient lookups of data, but it wouldn't ordinarily allow easy data purging.

How can we get the best of both worlds? By merging the two data structures. Here's how this works:

Just as before, we create a linked list where a node is moved to the front every time it's accessed. This way, the end of the linked list will always contain the stalest information.

In addition, we have a hash table that maps from a query to the corresponding node in the linked list. This allows us to not only efficiently return the cached results, but also to move the appropriate node to the front of the list, thereby updating its "freshness."

For illustrative purposes, abbreviated code for the cache is below. The code attachment provides the full code for this part. Note that in your interview, it is unlikely that you would be asked to write the full code for this as well as perform the design for the larger system.

```
1 public class Cache {
2     public static int MAX_SIZE = 10;
3     public Node head, tail;
4     public HashMap<String, Node> map;
5     public int size = 0;
6
7     public Cache() {
8         map = new HashMap<String, Node>();
9     }
10
11     /* Moves node to front of linked list */
12     public void moveToFront(Node node) { ... }
13     public void moveToFront(String query) { ... }
14
15     /* Removes node from linked list */
16     public void removeFromLinkedList(Node node) { ... }
17
18     /* Gets results from cache, and updates linked list */
19     public String[] getResults(String query) {
20         if (!map.containsKey(query)) return null;
21
22         Node node = map.get(query);
23         moveToFront(node); // update freshness
24         return node.results;
25     }
26
27     /* Inserts results into linked list and hash */
28     public void insertResults(String query, String[] results) {
29         if (map.containsKey(query)) { // update values
30             Node node = map.get(query);
31             node.results = results;
32             moveToFront(node); // update freshness
33             return;
34         }
35
36         Node node = new Node(query, results);
37         moveToFront(node);
38         map.put(query, node);
39
40         if (size > MAX_SIZE) {
41             map.remove(tail.query);
42             removeFromLinkedList(tail);
43         }
44     }
45 }
```

```
44     }  
45 }
```

Step 2: Expand to Many Machines

Now that we understand how to design this for a single machine, we need to understand how we would design this when queries could be sent to many different machines. Recall from the problem statement that there's no guarantee that a particular query will be consistently sent to the same machine.

The first thing we need to decide is to what extent the cache is shared across machines. We have several options to consider.

Option 1: Each machine has its own cache.

A simple option is to give each machine its own cache. This means that if "foo" is sent to machine 1 twice in a short amount of time, the result would be recalled from the cache on the second time. But, if "foo" is sent first to machine 1 and then to machine 2, it would be treated as a totally fresh query both times.

This has the advantage of being relatively quick, since no machine-to-machine calls are used. The cache, unfortunately, is somewhat less effective as an optimization tool as many repeat queries would be treated as fresh queries.

Option 2: Each machine has a copy of the cache.

On the other extreme, we could give each machine a complete copy of the cache. When new items are added to the cache, they are sent to all machines. The entire data structure—linked list and hash table—would be duplicated.

This design means that common queries would nearly always be in the cache, as the cache is the same everywhere. The major drawback however is that updating the cache means firing off data to N different machines, where N is the size of the response cluster. Additionally, because each item effectively takes up N times as much space, our cache would hold much less data.

Option 3: Each machine stores a segment of the cache.

A third option is to divide up the cache, such that each machine holds a different part of it. Then, when machine i needs to look up the results for a query, machine i would figure out which machine holds this value, and then ask this other machine (machine j) to look up the query in j 's cache.

But how would machine i know which machine holds this part of the hash table?

One option is to assign queries based on the formula $\text{hash}(\text{query}) \% N$. Then, machine i only needs to apply this formula to know that machine j should store the results for this query.

So, when a new query comes in to machine i , this machine would apply the formula and call out to machine j . Machine j would then return the value from its cache or call `processSearch(query)` to get the results. Machine j would update its cache and return the results back to i .

Alternatively, you could design the system such that machine j just returns `null` if it doesn't have the query in its current cache. This would require machine i to call `processSearch` and then forward the results to machine j for storage. This implementation actually increases the number of machine-to-machine calls, with few advantages.

Step 3: Updating results when contents change

Recall that some queries may be so popular that, with a sufficiently large cache, they would permanently be cached. We need some sort of mechanism to allow cached results to be refreshed, either periodically or “on-demand” when certain content changes.

To answer this question, we need to consider when results would change (and you need to discuss this with your interviewer). The primary times would be when:

1. The content at a URL changes (or the page at that URL is removed).
2. The ordering of results change in response to the rank of a page changing.
3. New pages appear related to a particular query.

To handle situations #1 and #2, we could create a separate hash table that would tell us which cached queries are tied to a specific URL. This could be handled completely separately from the other caches, and reside on different machines. However, this solution may require a lot of data.

Alternatively, if the data doesn’t require instant refreshing (which it probably doesn’t), we could periodically crawl through the cache stored on each machine to purge queries tied to the updated URLs.

Situation #3 is substantially more difficult to handle. We could update single word queries by parsing the content at the new URL and purging these one-word queries from the caches. But, this will only handle the one-word queries.

A good way to handle Situation #3 (and likely something we’d want to do anyway) is to implement an “auto-matic time-out” on the cache. That is, we’d impose a time out where *no* query, regardless of how popular it is, can sit in the cache for more than *x* minutes. This will ensure that all data is periodically refreshed.

Step 4: Further Enhancements

There are a number of improvements and tweaks you could make to this design depending on the assumptions you make and the situations you optimize for.

One such optimization is to better support the situation where some queries are very popular. For example, suppose (as an extreme example) a particular string constitutes 1% of all queries. Rather than machine *i* forwarding the request to machine *j* every time, machine *i* could forward the request just once to *j*, and then *i* could store the results in its own cache as well.

Alternatively, there may also be some possibility of doing some sort of re-architecture of the system to assign queries to machines based on their hash value (and therefore the location of the cache), rather than randomly. However, this decision may come with its own set of trade-offs.

Another optimization we could make is to the “automatic time out” mechanism. As initially described, this mechanism purges any data after *X* minutes. However, we may want to update some data (like current news) much more frequently than other data (like historical stock prices). We could implement timeouts based on topic or based on URLs. In the latter situation, each URL would have a time out value based on how frequently the page has been updated in the past. The time out for the query would be the minimum of the time outs for each URL.

These are just a few of the enhancements we can make. Remember that in questions like this, there is no single correct way to solve the problem. These questions are about having a discussion with your interviewer about design criteria and demonstrating your general approach and methodology.

9.6 Sales Rank: A large eCommerce company wishes to list the best-selling products, overall and by category. For example, one product might be the #1056th best-selling product overall but the #13th best-selling product under "Sports Equipment" and the #24th best-selling product under "Safety." Describe how you would design this system.

pg 145

SOLUTION

Let's first start off by making some assumptions to define the problem.

Step 1: Scope the Problem

First, we need to define what exactly we're building.

- We'll assume that we're only being asked to design the components relevant to this question, and not the entire eCommerce system. In this case, we might touch the design of the frontend and purchase components, but only as it impacts the sales rank.
- We should also define what the sales rank means. Is it total sales over all time? Sales in the last month? Last week? Or some more complicated function (such as one involving some sort of exponential decay of sales data)? This would be something to discuss with your interviewer. We will assume that it is simply the total sales over the past week.
- We will assume that each product can be in multiple categories, and that there is no concept of "subcategories."

This part just gives us a good idea of what the problem, or scope of features, is.

Step 2: Make Reasonable Assumptions

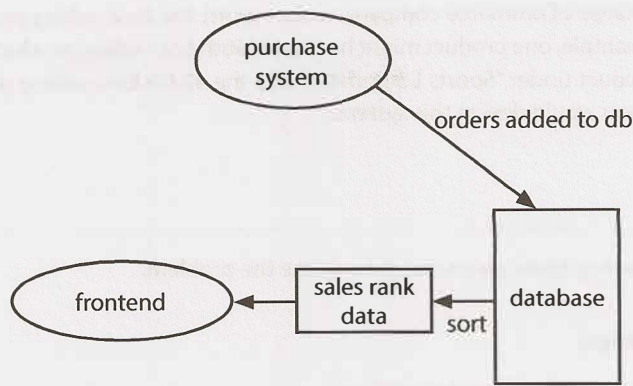
These are the sorts of things you'd want to discuss with your interviewer. Because we don't have an interviewer in front of us, we'll have to make some assumptions.

- We will assume that the stats do not need to be 100% up-to-date. Data can be up to an hour old for the most popular items (for example, top 100 in each category), and up to one day old for the less popular items. That is, few people would care if the #2,809,132th best-selling item should have actually been listed as #2,789,158th instead.
- Precision is important for the most popular items, but a small degree of error is okay for the less popular items.
- We will assume that the data should be updated every hour (for the most popular items), but the time range for this data does not need to be precisely the last seven days (168 hours). If it's sometimes more like 150 hours, that's okay.
- We will assume that the categorizations are based strictly on the origin of the transaction (i.e., the seller's name), not the price or date.

The important thing is not so much which decision you made at each possible issue, but whether it occurred to you that these are assumptions. We should get out as many of these assumptions as possible in the beginning. It's possible you will need to make other assumptions along the way.

Step 3: Draw the Major Components

We should now design just a basic, naive system that describes the major components. This is where you would go up to a whiteboard.



In this simple design, we store every order as soon as it comes into the database. Every hour or so, we pull sales data from the database by category, compute the total sales, sort it, and store it in some sort of sales rank data cache (which is probably held in memory). The frontend just pulls the sales rank from this table, rather than hitting the standard database and doing its own analytics.

Step 4: Identify the Key Issues

Analytics are Expensive

In the naive system, we periodically query the database for the number of sales in the past week for each product. This will be fairly expensive. That’s running a query over all sales for all time.

Our database just needs to track the total sales. We’ll assume (as noted in the beginning of the solution) that the general storage for purchase history is taken care of in other parts of the system, and we just need to focus on the sales data analytics.

Instead of listing every purchase in our database, we’ll store just the total sales from the last week. Each purchase will just update the total weekly sales.

Tracking the total sales takes a bit of thought. If we just use a single column to track the total sales over the past week, then we’ll need to re-compute the total sales every day (since the specific days covered in the last seven days change with each day). That is unnecessarily expensive.

Instead, we’ll just use a table like this.

Prod ID	Total	Sun	Mon	Tues	Wed	Thurs	Fri	Sat

This is essentially like a circular array. Each day, we clear out the corresponding day of the week. On each purchase, we update the total sales count for that product on that day of the week, as well as the total count.

We will also need a separate table to store the associations of product IDs and categories.

Prod ID	Category ID

To get the sales rank per category, we’ll need to join these tables.

Database Writes are Very Frequent

Even with this change, we'll still be hitting the database very frequently. With the amount of purchases that could come in every second, we'll probably want to batch up the database writes.

Instead of immediately committing each purchase to the database, we could store purchases in some sort of in-memory cache (as well as to a log file as a backup). Periodically, we'll process the log / cache data, gather the totals, and update the database.

We should quickly think about whether or not it's feasible to hold this in memory. If there are 10 million products in the system, can we store each (along with a count) in a hash table? Yes. If each product ID is four bytes (which is big enough to hold up to 4 billion unique IDs) and each count is four bytes (more than enough), then such a hash table would only take about 40 megabytes. Even with some additional overhead and substantial system growth, we would still be able to fit this all in memory.

After updating the database, we can re-run the sales rank data.

We need to be a bit careful here, though. If we process one product's logs before another's, and re-run the stats in between, we could create a bias in the data (since we're including a larger timespan for one product than its "competing" product).

We can resolve this by either ensuring that the sales rank doesn't run until all the stored data is processed (difficult to do when more and more purchases are coming in), or by dividing up the in-memory cache by some time period. If we update the database for all the stored data up to a particular moment in time, this ensures that the database will not have biases.

Joins are Expensive

We have potentially tens of thousands of product categories. For each category, we'll need to first pull the data for its items (possibly through an expensive join) and then sort those.

Alternatively, we could just do one join of products and categories, such that each product will be listed once per category. Then, if we sorted that on category and then product ID, we could just walk the results to get the sales rank for each category.

Prod ID	Category	Total	Sun	Mon	Tues	Wed	Thurs	Fri	Sat
1423	sportseq	13	4	1	4	19	322	32	232
1423	safety	13	4	1	4	19	322	32	232

Rather than running thousands of queries (one for each category), we could sort the data on the category first and then the sales volume. Then, if we walked those results, we would get the sales rank for each category. We would also need to do one sort of the entire table on just sales number, to get the overall rank.

We could also just keep the data in a table like this from the beginning, rather than doing joins. This would require us to update multiple rows for each product.

Database Queries Might Still Be Expensive

Alternatively, if the queries and writes get very expensive, we could consider forgoing a database entirely and just using log files. This would allow us to take advantage of something like MapReduce.

Under this system, we would write a purchase to a simple text file with the product ID and time stamp. Each category has its own directory, and each purchase gets written to all the categories associated with that product.

We would run frequent jobs to merge files together by product ID and time ranges, so that eventually all purchases in a given day (or possibly hour) were grouped together.

```
/sportsequipment
1423,Dec 13 08:23-Dec 13 08:23,1
4221,Dec 13 15:22-Dec 15 15:45,5
...
/safety
1423,Dec 13 08:23-Dec 13 08:23,1
5221,Dec 12 03:19-Dec 12 03:28,19
...
```

To get the best-selling products within each category, we just need to sort each directory.

How do we get the overall ranking? There are two good approaches:

- We could treat the general category as just another directory, and write every purchase to that directory. That would mean a lot of files in this directory.
- Or, since we'll already have the products sorted by sales volume order for each category, we can also do an N-way merge to get the overall rank.

Alternatively, we can take advantage of the fact that the data doesn't need (as we assumed earlier) to be 100% up-to-date. We just need the most popular items to be up-to-date.

We can merge the most popular items from each category in a pairwise fashion. So, two categories get paired together and we merge the most popular items (the first 100 or so). After we have 100 items in this sorted order, we stop merging this pair and move onto the next pair.

To get the ranking for all products, we can be much lazier and only run this work once a day.

One of the advantages of this is that it scales nicely. We can easily divide up the files across multiple servers, as they aren't dependent on each other.

Follow Up Questions

The interviewer could push this design in any number of directions.

- Where do you think you'd hit the next bottlenecks? What would you do about that?
- What if there were subcategories as well? So items could be listed under "Sports" and "Sports Equipment" (or even "Sports" > "Sports Equipment" > "Tennis" > "Rackets")?
- What if data needed to be more accurate? What if it needed to be accurate within 30 minutes for all products?

Think through your design carefully and analyze it for the tradeoffs. You might also be asked to go into more detail on any specific aspect of the product.

- 9.7 Personal Financial Manager:** Explain how you would design a personal financial manager (like Mint.com). This system would connect to your bank accounts, analyze your spending habits, and make recommendations.

pg 145

SOLUTION

The first thing we need to do is define what it is exactly that we are building.

Step 1: Scope the Problem

Ordinarily, you would clarify this system with your interviewer. We'll scope the problem as follows:

- You create an account and add your bank accounts. You can add multiple bank accounts. You can also add them at a later point in time.
- It pulls in all your financial history, or as much of it as your bank will allow.
- This financial history includes outgoing money (things you bought or paid for), incoming money (salary and other payments), and your current money (what's in your bank account and investments).
- Each payment transaction has a "category" associated with it (food, travel, clothing, etc.).
- There is some sort of data source provided that tells the system, with some reliability, which category a transaction is associated with. The user might, in some cases, override the category when it's improperly assigned (e.g., eating at the cafe of a department store getting assigned to "clothing" rather than "food").
- Users will use the system to get recommendations on their spending. These recommendations will come from a mix of "typical" users ("people generally shouldn't spend more than X% of their income on clothing"), but can be overridden with custom budgets. This will not be a primary focus right now.
- We assume this is just a website for now, although we could potentially talk about a mobile app as well.
- We probably want email notifications either on a regular basis, or on certain conditions (spending over a certain threshold, hitting a budget max, etc.).
- We'll assume that there's no concept of user-specified rules for assigning categories to transactions.

This gives us a basic goal for what we want to build.

Step 2: Make Reasonable Assumptions

Now that we have the basic goal for the system, we should define some further assumptions about the characteristics of the system.

- Adding or removing bank accounts is relatively unusual.
- The system is write-heavy. A typical user may make several new transactions daily, although few users would access the website more than once a week. In fact, for many users, their primary interaction might be through email alerts.
- Once a transaction is assigned to a category, it will only be changed if the user asks to change it. The system will never reassign a transaction to a different category "behind the scenes", even if the rules change. This means that two otherwise identical transactions could be assigned to different categories if the rules changed in between each transaction's date. We do this because it may confuse users if their spending per category changes with no action on their part.
- The banks probably won't push data to our system. Instead, we will need to pull data from the banks.
- Alerts on users exceeding budgets probably do not need to be sent instantaneously. (That wouldn't be realistic anyway, since we won't get the transaction data instantaneously.) It's probably pretty safe for them to be up to 24 hours delayed.

It's okay to make different assumptions here, but you should explicitly state them to your interviewer.